

玩转 Linux&Git，从此环境配置不踩坑！

SPEIT 科创工作坊 第一期

Linux

Git

Python

VS Code

> 从零开始，跑通第一个真实项目 _

WSL：在 Windows 运行原生 Linux

Windows Subsystem for Linux (WSL2) Configuration

核心逻辑

Windows 无法直接运行 **Linux** 原生工具链。**WSL2** 提供了一个高性能的 **Linux 内核**，使两者共存。

优势： 无缝调用 **GPU (CUDA)**、完美支持 **Docker**、极速读写 **Linux** 文件系统。

常见痛点

- 网络下载极慢
- **localhost** 端口无法互通
- 无法使用 **Windows** 端的代理

推荐配置方案：镜像网络

在 **Windows** 侧创建文件：**C:\Users\<你的用户名>\.wslconfig**

```
[wsl2]
# 开启镜像网络模式 (共享 IP)
networkingMode=mirrored
# 开启 DNS 代理 (解决解析慢)
dnsProxy=true
# 自动回收空闲内存
autoMemoryReclaim=gradual
```

重启生效：

```
$ wsl --shutdown
```

提示： 配置后，**WSL** 将自动继承 **Windows** 的网络环境，包括代理设置。

macOS：原生 Unix 环境与安全警示

macOS Developer Setup & Environment Isolation

>_ Mac用的天然优势

macOS 底层基于 **Unix**。你的终端环境是原生的，**无需 WSL**。你可以直接运行绝大多数 **Linux** 命令。

核心逻辑： 系统环境归 **macOS** 管理，开发环境归用户管理。两者必须完全隔离。

👍 开发者首选原因：

- 完美兼容 **POSIX** 标准
- 无缝衔接各类编译工具链
- 文件读写性能极高

💀 禁区：系统 Python

macOS 系统自带一个 **Python** (位于 **/usr/bin/python**)。

严禁操作：

- 使用 **sudo pip install**
- 修改系统 **Python** 的任何库文件

后果： 系统 **Python** 是给 macOS 内部工具用的。一旦损坏，可能导致系统工具崩溃甚至无法启动。

包管理器：命令行“应用商店”

Package Managers: apt (Ubuntu) & brew (macOS)

>_ Ubuntu 常用命令 (apt)

通过命令行直接管理软件。无需手动下载安装包，自动处理依赖关系。

功能	命令	说明
刷新列表	<code>sudo apt update</code>	同步远程仓库的最新软件信息
安装软件	<code>sudo apt install git</code>	下载并安装指定软件包
卸载软件	<code>sudo apt remove git</code>	从系统中移除指定软件包
搜索软件	<code>apt search python</code>	在仓库中查找匹配的包名

 注意： `sudo` 表示以管理员权限运行，执行时需输入开机密码。

🔄 apt vs apt-get

apt 是现代推荐工具。它整合了 `apt-get` 和 `apt-cache` 的常用功能，命令更简洁，且提供进度条等更好的交互体验。

🍺 macOS (Homebrew)

Mac 同学请安装 **Homebrew**。它是 **macOS** 事实上缺失的包管理器。

```
$ brew install git
$ brew update
$ brew upgrade
```

VS Code：现代开发者的首选编辑器

Visual Studio Code: Lightweight, Extensible, and Powerful

★ 核心优势

VS Code 是目前最流行的开源编辑器。它兼具 **IDE** 的强大功能与文本编辑器的轻量体验。

- **极致扩展性**：拥有全球最大的插件生态。
- **完美远程开发**：唯一能无缝联动 **WSL/SSH** 的编辑器。
- **内置终端**：无需切换窗口即可运行命令。

↓ 安装指南

访问官网：code.visualstudio.com

- Windows：下载 .exe 安装包，一路 Next。
- macOS：下载 .zip，解压后将 App 拖入 Applications。

🧩 必装插件 (Extensions)



Python

提供代码补全、调试、**Lint**er 支持。



GitLens

可视化 **Git** 历史，查看每一行代码是谁写的。



Remote Development

包含 **WSL, SSH, Containers** 远程开发全家桶。

💡 提示：在 **VS Code** 左侧边栏点击方块图标即可搜索并安装插件。

WSL + VS Code：最顺滑的联动开发体验

Remote Development: Code in Windows, Run in Linux

🔌 核心插件：WSL

在 **VS Code** 插件市场搜索并安装 **WSL** (由 **Microsoft** 出品)。

原理： **VS Code** 会在 **WSL** 内部启动一个轻量级服务器。你的代码、编译器、终端都在 **Linux** 侧，但 **UI** 界面在 **Windows** 侧。

👁️ 识别连接状态

成功连接后，**VS Code** 左下角的状态栏会显示绿色图标：

↔️ WSL: Ubuntu-24.04

此时你打开的终端就是 **WSL** 终端，安装的插件也是安装在 **WSL** 内部的。

🔧 魔法命令：code .

这是最推荐的启动方式。在 **WSL** 终端中进入你的项目目录，然后输入：

```
$ cd ~/my-project  
$ code .
```

✅ 为什么这么做？

- **性能：** 避免了跨文件系统的读写延迟。
- **环境：** 插件直接识别 **Linux** 路径下的 **Python** 解释器。
- **体验：** 享受 **Windows** 的丝滑 **UI**，同时拥有 **Linux** 的强大底层。

VS Code Git (1): 基础操作可视化

Source Control Panel: Add, Commit, and Sync

1

暂存 (Add)

main.py



utils.py



点击文件旁的 + 号。
相当于执行 `git add`。
将修改放入“待提交”清单。

2

提交 (Commit)

输入提交信息...

提交 (Commit)

在输入框写下备注。
点击 **对勾** 或提交按钮。
相当于执行 `git commit`。

3

同步 (Sync)



同步更改

点击底部的 **循环图标**。
相当于 `pull + push`。
将本地代码推送到 **GitHub**。

提示： 点击 VS Code 左侧边栏的  图标即可进入“源代码管理”面板。

VS Code Git (2): GitLens 进阶神器用法

GitLens: Supercharge your Git workflow with Visual Insights

👁 让代码历史“透明化”

行内注释 (Current Line Blame)

鼠标停留在任何一行代码，末尾会淡淡显示：

You, 2 days ago • Initial commit

版本历史树 (Visual File History)

可视化查看该文件的所有提交记录，点击即可对比任意两个版本之间的差异。

💡 核心价值：无需敲 `git log`，直接在编辑器里看清代码的前世今生。

📊 可视化对比与追踪

1. 侧边栏对比 (Side-by-Side Diff)

```
- old_function()
- print("Hello")
```

```
+ new_function()
+ print("Hello World")
```

2. 提交图谱 (Commit Graph)

- feat: add login page (main)
- fix: header style
- initial commit

💡 提示：安装 **GitLens** 后，点击左侧 **Git** 图标，在下方找到 **FILE HISTORY** 即可开启上帝视角。

Ubuntu 24.04: 为什么 pip install 报错?

PEP 668: Marking Python Environments as Externally Managed

⚠️ 预期的报错信息

```
$ pip install numpy
error: externally-managed-environment

× This environment is externally managed
└> To install Python packages system-wide, try apt install
    python3-xyz ...
```

核心原因: PEP 668

Ubuntu 24.04+ 遵循 **PEP 668** 规范，禁止在系统 Python 环境中使用 **pip** 全局安装第三方库。

逻辑: 系统工具（如 **apt**, **ufw**）依赖特定版本的 **Python** 库。全局 **pip** 安装可能导致版本冲突，进而破坏系统核心组件。

✅ 正确方案

强制使用虚拟环境

这不是障碍，而是**工业界标准**。你必须为每个项目创建独立的隔离环境。

- 方案 A: 使用 **Conda** (推荐)
- 方案 B: 使用 **venv**
- 方案 C: 使用 **pipx** (仅限命令行工具)

系统安全 vs 开发隔离:

系统环境归 **apt** 管理；
开发环境归 **conda/pip** 管理。
两者互不干扰，确保系统永远稳定。

Conda 与 Pip：隔离环境黄金法则

Environment Management vs Package Installation

角色分工

Conda (环境管理): 负责创建隔离的虚拟环境。它管理 **Python** 版本及底层二进制依赖（如 **CUDA, C++** 库）。

Pip (包安装): 负责在已激活的环境中安装 **Python** 库。它只管理 **Python** 内部的包。

为什么要隔离？

不同项目对库的版本要求不同。隔离环境能确保项目 **A** 的更新不会破坏项目 **B** 的运行。

现代开发黄金法则

Conda 建环境 → Pip 装包

这是目前科研与工程界最稳健的实践方案。

警告：严禁混用

在同一环境频繁交替使用 `conda install` 和 `pip install` 会导致依赖元数据冲突。一旦环境崩溃，通常无法修复，只能删除重来。

提速：配置国内镜像源 (pip/conda/ubuntu)

Mirrors Configuration for Faster Downloads (WSL & Mac)

1. pip 换源

```
# 永久修改全局下载源 (清华源)
$ pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple

# 验证配置
$ pip config list
```

2. conda 换源

```
# 添加清华镜像 Channels
$ conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
$ conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
$ conda config --set show_channel_urls yes
```

3. 系统源适配

Ubuntu 24.04 (WSL)

```
# 24.04 采用新版 .sources 格式
$ sudo sed -i 's|http://archive.ubuntu.com|https://mirrors.tuna.tsinghua.edu.cn|g' /etc/apt/sources.list.d/ubuntu.sources
$ sudo apt update
```

macOS (Homebrew)

```
# 替换 brew 核心仓库
$ git -C "$(brew --repo)" remote set-url origin https://mirrors.tuna.tsinghua.edu.cn/git/homebrew/brew.git
```

⚡ **核心逻辑：** 默认下载地址位于海外。换源可将“跨境访问”转为“本地内网”，下载速度通常提升 **10-100** 倍。

连接 GitHub：SSH 免密方案

Connecting to GitHub: HTTPS vs SSH & Setup Guide

为什么推荐用 SSH?

HTTPS 访问： 每次 **push** 代码都要输入 **Personal Access Token**，繁琐且难以管理。

SSH 访问： 基于非对称加密。配置一次后，这台电脑访问 **GitHub** 将永久免密，安全且高效。

💡 核心逻辑：

本地生成密钥对，将公钥上传至 **GitHub**。下次访问时，**GitHub** 通过非对称加密自动完成身份验证。

3 步搞定 SSH 配置

Step 1: 生成密钥对 (一路回车即可)

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

Step 2: 查看并复制公钥内容

```
$ cat ~/.ssh/id_ed25519.pub
```

(复制以 `ssh-ed25519` 开头的一整行字符)

Step 3: 贴入 GitHub 设置

GitHub → Settings → **SSH and GPG keys** → New SSH Key → 粘贴公钥内容并保存。

Git 加速

Git Acceleration: Reliable Proxy & WSL Mirrored Network

🛠️ 方案 A: Git 全局代理

如果你电脑上运行着代理软件（如 **Clash/V2Ray**），直接让 **Git** 走本地代理端口。

```
# 设置 HTTP/HTTPS 代理 (假设端口为 7890)
$ git config --global http.proxy http://127.0.0.1:7890
$ git config --global https.proxy http://127.0.0.1:7890

# 取消代理设置
$ git config --global --unset http.proxy
$ git config --global --unset https.proxy
```

🖥️ 方案 B: WSL 镜像网络

这是 **WSL 用户** 的终极方案。开启 `networkingMode=mirrored` 后，**WSL** 会自动继承 **Windows** 的网络环境。

优势： 无需在 **Linux** 内部重复配置代理，**Git** 会自动通过 **Windows** 端的代理软件出海。

逻辑： 镜像网络让 **WSL** 共享 **Windows** 的 IP 栈，所有网络流量对代理软件透明。

⚠️ **避坑指南：** 弃用修改 **Hosts** 或使用第三方镜像站。这些方法极不稳定，且存在安全风险。

课堂实战：运行第一个项目 (Streamlit Demo)

Full Workflow: From Environment Setup to Running the App

1 创建并激活环境

```
$ conda create -n workshop python=3.10 -y  
$ conda activate workshop
```

2 克隆项目代码

```
$ git clone git@github.com:example/workshop-demo.git  
$ cd workshop-demo
```

3 安装项目依赖

```
$ pip install -r requirements.txt
```

4 启动应用

```
$ streamlit run app.py
```

🚀 看到浏览器自动弹出页面，恭喜你，环境配置大功告成！

总结：解决问题是唯一目的

终极绝招：删掉重来

环境配置最忌讳“缝缝补补”。如果一个报错折腾了 30 分钟还没解决，请果断执行以下命令，从头开始。这通常是最快、最稳妥的方案。

```
$ conda deactivate  
$ conda remove -n my_env --all -y
```



1. 读报错

报错信息通常在最后 3 行。
看清是 **Network** 问题、
Permission 问题还是 **Version** 冲突。



2. 搜 Google

直接复制报错的第一行。
优先看 **StackOverflow** 或
GitHub Issues。



3. 问 AI / 学长

提问时可以附带：

1. 你的操作系统
2. 你执行的完整命令
3. 完整的报错截图